IE3142 · DEVOPS SECURITY

Secure Continuous Integration and Deployment

Lecture 06 —Integrating security into the delivery pipeline

LECTURE ROADMAP

Topics Covered

1. Introduction to Containerization
2. Containers vs Virtual Machines
3. Docker Architecture
4. Container Security Challenges
5. Container Security Best Practices
6. Container Image Security
7. Container Runtime Security
8. Introduction to Kubernetes
9. Kubernetes Architecture
10. Kubernetes Security
11. Secure Kubernetes Configuration
12. Container Security in DevSecOps

What is CI/CD?

Continuous Integration (CI): frequently integrating code changes into a shared repository and automatically building and testing the application.

- Code compilation
- Unit testing
- Integration testing
- Code quality analysis
- Security scanning

Continuous Delivery (CD): prepares validated software for release.
Continuous Deployment (CD): automatically deploys validated changes to production.

Typical Pipeline



Why Integrate Security into CI/CD?

Traditional security testing happens near the end of development, vulnerabilities are discovered late, forcing teams back to earlier stages.

Traditional Approach

Code → Build → Test → Deploy

→ Security Testing

→ **Vulnerability Found**

DevSecOps Approach

Code → Security Checks → Build

→ Security Tests → Deploy

→ Continuous Monitoring

Benefits

- Earlier vulnerability detection
- Automated security testing
- Lower remediation cost
- Consistent security controls
- Faster feedback
- Reduced human error

Security in the CI/CD Pipeline

Stage	Security Activity
Code	SAST, secret scanning
Dependencies	SCA
Build	Secure build validation
Container	Image scanning
Testing	DAST, security tests
Infrastructure	IaC scanning
Deployment	Policy checks
Runtime	Monitoring and threat detection

Shift-Left Security in CI/CD

Shift Left means moving security activities earlier in the software development lifecycle.

Traditional — too late

Development → Testing → Deployment → Security

Shift Left

Security → Development → Testing → Deployment

Example — catching a vulnerable dependency early

Developer adds dependency → SCA Scan → Vulnerability detected → Developer fixes it

This reduces the cost and effort of remediation.

Static Application Security Testing (SAST)

SAST analyzes source code or compiled code without executing the application.

SAST Can Identify:

- SQL Injection
- Hardcoded credentials
- Weak cryptography
- Insecure functions
- Input validation issues

Example Tools

SonarQube · Semgrep · Checkmarx · Fortify



Software Composition Analysis (SCA)

Modern applications depend heavily on third-party libraries. SCA identifies vulnerabilities in these dependencies.

Application

```
├── Flask
├── Requests
├── OpenSSL
├── Other Libraries
```

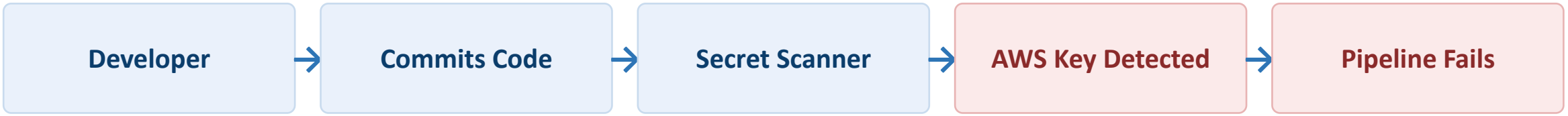
SCA Checks

- Dependency versions
- Known vulnerabilities
- Outdated packages
- License information
- Dependency relationships

Example Tools: OWASP Dependency-Check · Snyk · Dependabot

Secret Scanning

Sensitive credentials can accidentally be committed to source-code repositories; AWS access keys, API tokens, database passwords, private keys.



Best Practices

- Never hard-code secrets
- Scan repositories automatically
- Rotate exposed credentials immediately
- Use secret-management systems

Example Tools

- **GitHub Secret Scanning**
- **Gitleaks**
- **TruffleHog**

Dynamic Application Security Testing (DAST)

DAST tests a running application from the outside to identify vulnerabilities. Unlike SAST, DAST does not analyze source code directly.

DAST Can Identify:

- SQL Injection
- Cross-Site Scripting
- Authentication weaknesses
- Session management problems
- Security misconfigurations

Example Tool: OWASP ZAP



Infrastructure as Code Security

Modern DevOps environments use Infrastructure as Code (IaC) to define infrastructure through configuration files, Terraform, AWS CloudFormation, Azure Bicep.



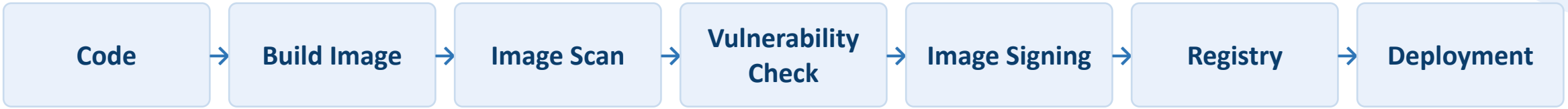
IaC Scanning Can Identify

- Public storage
- Open firewall rules
- Excessive permissions
- Unencrypted resources
- Insecure network configurations

Example Tools

- Checkov
- tfsec
- KICS

Container Security in CI/CD



Security Checks

- Scan image vulnerabilities
- Scan dependencies
- Detect secrets
- Verify base images
- Check Dockerfile security
- Generate SBOM
- Verify image signatures

Pipeline Decision

Security Scan

Critical Vulnerability

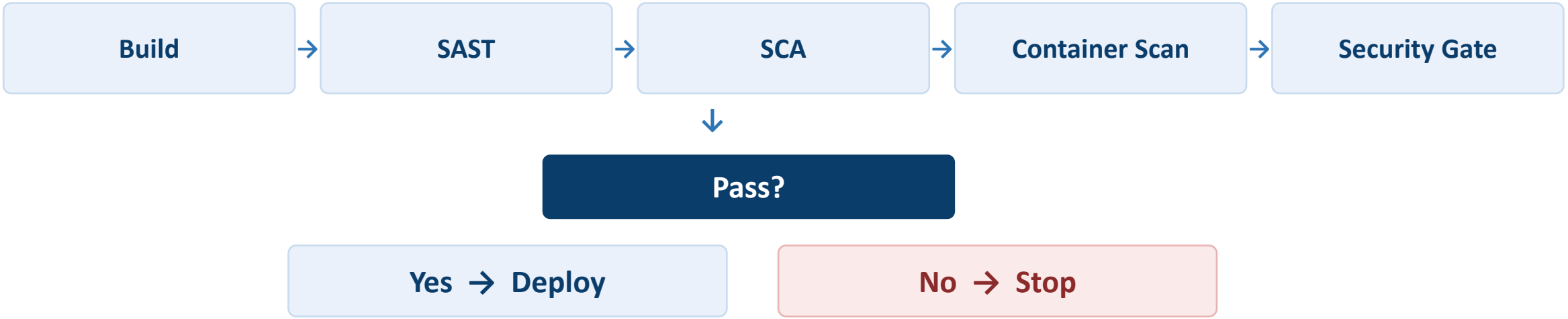
Pipeline FAILED

Deployment BLOCKED

This prevents vulnerable container images from reaching production.

Security Gates in CI/CD

A security gate is an automated condition in a CI/CD pipeline that determines whether a build can proceed to the next stage.



Example Policy

- No Critical vulnerabilities
- No exposed secrets
- Maximum 5 Medium vulnerabilities
- No high-risk IaC misconfigurations

Security Automation in CI/CD

1

Automated Code Scanning

Every code commit is scanned.

2

Automated Dependency Scanning

New dependencies are checked for vulnerabilities.

3

Automated Container Scanning

Container images are scanned before deployment.

4

Automated Infrastructure Scanning

Configurations checked before provisioning.

5

Automated Compliance

Checked against security policies.

CI/CD Pipeline Security

Pipeline Security Threats

- Compromised developer credentials
- Malicious code commits
- Compromised build servers
- Stolen CI/CD tokens
- Malicious dependencies
- Unauthorized pipeline modifications
- Supply-chain attacks

Security Controls

- Strong authentication & MFA
- Least privilege
- Protected branches
- Code review
- Secure build agents
- Secret management
- Artifact signing
- Audit logging

Software Supply Chain Security

Modern applications are built using many external components, each dependency introduces potential security risk.

Application

```
├── Open Source Libraries
├── Container Images
├── Build Tools
├── APIs
├── Third-Party Services
```

Attackers May Compromise

- Open-source packages
- Container images
- Build systems
- CI/CD pipelines
- Package repositories

Security Measures

- Dependency scanning
- Artifact verification
- SBOM
- Version pinning
- Trusted repositories
- Continuous monitoring
- Image signing

SUPPLY CHAIN

Secure Artifact Management

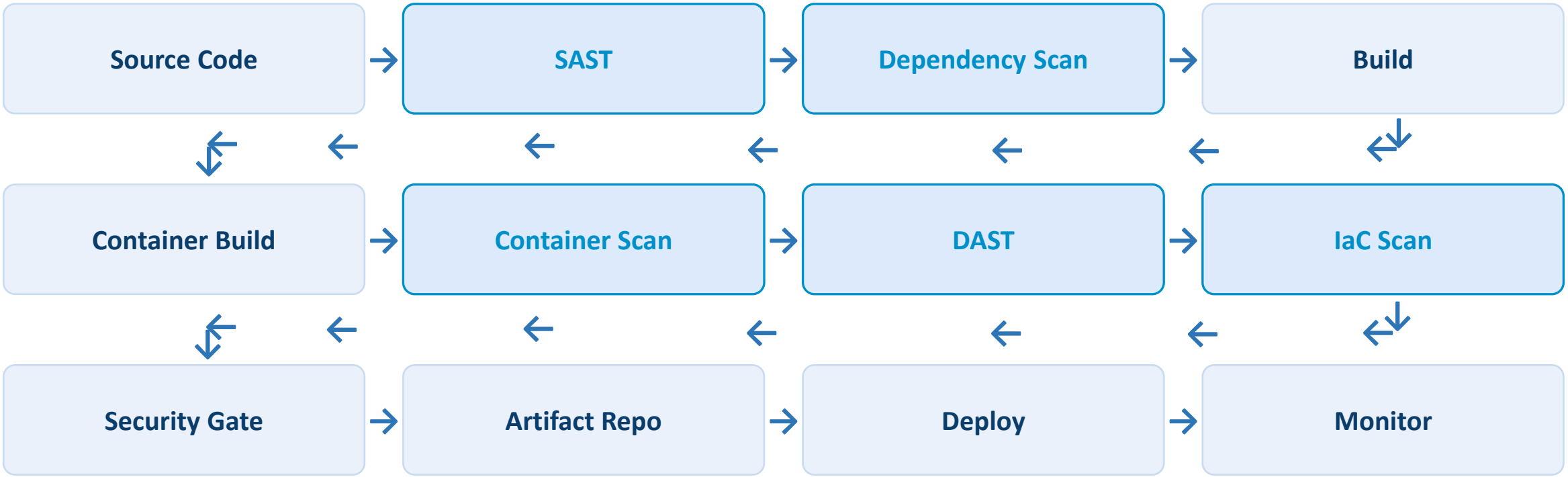
An artifact is a file produced during the software build process executables, packages, container images, libraries, and deployment packages.

Artifacts should be securely stored and verified before deployment.

Best Practices

- Use trusted artifact repositories
- Scan artifacts
- Verify integrity
- Prevent unauthorized modification
- Control access
- Sign artifacts
- Maintain version history

DevSecOps CI/CD Pipeline



Cyan boxes are security activities. Security is continuously integrated rather than performed as a final stage.

WORKED EXAMPLE

Example: Secure CI/CD Pipeline for an Online Shopping App

1. **Code**
   Secret scanning checks for exposed credentials.

2. **SAST**
   Source code is analyzed for vulnerabilities.

3. **SCA**
   Third-party dependencies are scanned.

4. **Build**
   The application is packaged into a container.

5. **Container Scan**
   The container image is checked for vulnerabilities.

6. **IaC Scan**
   Terraform configuration is checked for misconfigurations.

7. **Security Gate**
   The pipeline checks whether security policies are satisfied.

8. **Deployment**
   Only approved artifacts are deployed.

9. **Runtime Monitoring**
   The application and infrastructure are continuously monitored.

Benefits of Secure CI/CD

1

Earlier Detection

Vulnerabilities identified earlier.

2

Lower Remediation Cost

Fix issues before deployment.

3

Faster Feedback

Automated tools provide immediate feedback.

4

Consistency

Same checks applied to every build.

5

Reduced Human Error

Automation reduces manual reliance.

6

Continuous Compliance

Security policies enforced continuously.

7

Improved Collaboration

Dev, security, and ops share responsibility.

KEY TAKEAWAYS

Container, Kubernetes & CI/CD Security

Container Security

- Portability, consistency, isolation
- Not automatically secure
- Secure images, registries, runtime, hosts
- Apply least privilege
- Scan images and dependencies
- Protect secrets · monitor runtime

Kubernetes Security

- Secure the control plane
- Apply RBAC and least privilege
- Secure pods and containers
- Use network policies
- Protect secrets
- Avoid privileged workloads

Secure CI/CD

- Integrate security throughout the pipeline
- Use SAST, SCA, DAST, IaC & container scanning
- Scan for secrets
- Use security gates
- Protect the CI/CD pipeline & supply chain
- Automate security wherever possible

DEVSECOPS

Questions?